

Software de Orquestación

Terraform / OpenTofu

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI · Proyecto Intermodular

01

Infraestructura como Código

Qué es y por qué la usamos

Del despliegue manual al código

DESPLIEGUE TRADICIONAL

- Configuración manual, servidor a servidor
- **Difícil de reproducir** y de documentar
- La documentación se desactualiza o no existe
- *"Funciona en mi máquina"*

CON INFRAESTRUCTURA COMO CÓDIGO

- La infraestructura se **describe en ficheros de texto**
- Se crea y se modifica de forma **automática**
- Es **reproducible**: el mismo código da siempre el mismo resultado
- Se gestiona con **Git**, igual que el software

¿Qué es la Infraestructura como Código (IaC)?

" *Consiste en **gestionar y aprovisionar infraestructura** (servidores, redes, almacenamiento...) mediante **ficheros de código**, en lugar de configurarla manualmente.*

LO HACE POSIBLE

Los servicios de virtualización y Cloud Computing son **software**, y todo software se puede **programar mediante una API**.

BENEFICIOS

- Escenarios **replicables** y predecibles
- **Control de versiones** de toda la infraestructura
- **Auditoría** de los cambios y recuperación rápida ante desastres

Dos tipos de herramientas IaC



SOFTWARE DE ORQUESTACIÓN

Crea y gestiona **escenarios completos**:
máquinas virtuales, redes, almacenamiento...

Ejemplos: Terraform, OpenTofu, Pulumi,
CloudFormation, Heat



SOFTWARE DE GESTIÓN DE LA CONFIGURACIÓN (CMS)

Configura el **software** de las máquinas ya
creadas.

Ejemplos: Ansible, Puppet, Chef, Salt



En este módulo usamos **OpenTofu** para orquestar y **Ansible** para configurar: primero se crea la infraestructura, después se configura.

DevOps

Conflicto histórico: los equipos de Desarrollo (Dev) y de Sistemas (Ops) han tenido objetivos y herramientas distintas.

¿CÓMO LO SOLUCIONA DEVOPS?

- Mismas herramientas para dev y ops
- Extiende las buenas prácticas de desarrollo (Git, revisión de código, tests) a los sistemas
- Integración → entrega → despliegue **continuos**

¿TIENE RELACIÓN CON IAC?

Sí, es una de sus piedras angulares.
Sin IaC no se puede automatizar el aprovisionamiento de infraestructura, y sin automatización no hay DevOps.

PI · UNIDAD 1 — SOFTWARE DE ORQUESTACIÓN

02

Software de Orquestación

Crear infraestructura como código

¿Qué hace un software de orquestación?

Crea escenarios completos con múltiples servidores, redes o contenedores (aprovisionamiento de recursos).

- Útil con demanda **variable** de recursos
- Útil cuando la configuración **cambia continuamente**
- Puede incluir **autoescalado** y respuesta a eventos

ENFOQUE DECLARATIVO

En lugar de describir *cómo* crear la infraestructura, **declaramos el estado deseado** y la herramienta se encarga de alcanzarlo.

```
resource "libvirt_domain" "web" {  
  name    = "servidor-web"  
  memory = 1024  
  vcpu    = 1  
}
```


PI · UNIDAD 1 — SOFTWARE DE ORQUESTACIÓN

03

OpenTofu

Orquestación de infraestructura declarativa

De Terraform a OpenTofu

TERRAFORM

Herramienta de orquestación creada por **HashiCorp** (2014), lenguaje declarativo propio **HCL**. En agosto de 2023 cambió su licencia a **BUSL 1.1**, que ya no es software libre.

OPENTOFU

Fork libre de Terraform 1.5.x creado por la comunidad y gobernado por la **Linux Foundation**. Licencia **MPL 2.0** (libre), compatible con los ficheros `.tf` y con los providers de Terraform.



Usamos **OpenTofu** en lugar de Terraform precisamente por ser software libre, sin restricciones de uso.

Elementos de configuración

LOS CUATRO BLOQUES BÁSICOS

- **Provider:** plugin que conecta con una API concreta
- **Resource:** elemento de infraestructura a crear (VM, red, volumen...)
- **Variable:** parámetro para reutilizar la configuración
- **Output:** valor de salida (IP, ID...) tras aplicar el plan

EJEMPLO

```
variable "memoria" { default = 1024 }

resource "libvirt_domain" "web" {
  name     = "web"
  memory   = var.memoria
}

output "nombre" {
  value = libvirt_domain.web.name
}
```

Un mismo lenguaje, distintos providers

Un **provider** es un plugin que conecta OpenTofu con una plataforma concreta. Cada uno define sus propios *resources*, pero el lenguaje (HCL) es siempre el mismo — en este módulo usaremos `libvirt` y `openstack`.

PROVIDER	PLATAFORMA
<code>aws</code>	Amazon Web Services
<code>azurerm</code>	Microsoft Azure
<code>google</code>	Google Cloud Platform
<code>openstack</code>	Nube privada OpenStack
<code>libvirt</code>	Máquinas virtuales con KVM
<code>docker</code>	Contenedores Docker

Ficheros de un proyecto

LOS ESCRIBIMOS NOSOTROS

- `provider.tf` — el provider y su versión
- `variables.tf` — las variables reutilizables
- `main.tf` — los recursos: VMs, discos...
- `network.tf` — las redes
- `output.tf` — lo que se muestra al terminar (IPs...)
- `cloud-init/` — la configuración inicial de cada VM

LO GENERA OPENTOFU

- `terraform.tfstate` — el **estado** de la infraestructura
- `.terraform/` — providers descargados por `init`
- `.terraform.lock.hcl` — versiones exactas de los providers

No se editan a mano ni se suben a Git (`.gitignore`).

Dependencias entre recursos

```
resource "libvirt_volume" "disco" {  
  name = "web.qcow2"  
  ...  
}  
  
resource "libvirt_domain" "web" {  
  disk { volume_id = libvirt_volume.disco.id } # referencia a otro recurso  
}
```

- Al usar `libvirt_volume.disco.id`, OpenTofu sabe que **el disco va antes** que la máquina
- Con esas referencias construye un **grafo de dependencias**: crea en orden, lo independiente a la vez, y **destruye al revés**
- No importa en qué orden se escriban los recursos ni en qué fichero: se leen todos los `.tf` del directorio

El estado de la infraestructura

" El **estado** (`terraform.tfstate`) es un fichero donde OpenTofu guarda **qué recursos ha creado realmente** y con **qué identificadores**.

¿POR QUÉ SE GUARDA?

- Es la forma de saber **qué existe ya** sin preguntar a la plataforma recurso a recurso
- Permite **comparar** lo declarado en el código con lo que hay realmente creado

CUIDADO CON EL ESTADO

- **No se edita a mano**
- Si trabajan varios a la vez, hace falta un **backend remoto** compartido
- Si el estado se pierde, OpenTofu "olvida" lo que había creado

Comandos más útiles

COMANDO	DESCRIPCIÓN
<code>tofu init</code>	Inicializa el proyecto, descarga los <i>providers</i>
<code>tofu plan</code>	Muestra qué acciones se realizarán (sin aplicar)
<code>tofu apply</code>	Aplica los cambios: crea, modifica o elimina recursos
<code>tofu destroy</code>	Elimina todos los recursos gestionados por el proyecto
<code>tofu validate</code>	Verifica que la sintaxis de los ficheros <code>.tf</code> sea correcta
<code>tofu show</code>	Muestra el estado actual de los recursos creados
<code>tofu output</code>	Muestra los valores definidos en los bloques <code>output</code>
<code>tofu refresh</code>	Actualiza el estado con lo que hay realmente en la plataforma

Flujo de trabajo habitual

```
tofu init      # 1. Una sola vez: descarga el provider
tofu plan      # 2. Revisar qué se va a crear, cambiar o borrar
tofu apply     # 3. Crear el escenario (pide confirmación)
tofu output    # 4. Ver la información del output (IPs...)
tofu destroy   # 5. Eliminar todo el escenario
```

- Si cambias los ficheros, vuelve a `plan` y `apply`: OpenTofu solo aplica **la diferencia**
- Todos los comandos se ejecutan **en el directorio del proyecto**

¿Qué hace realmente `tofu plan`?

" `plan` *compara tres cosas y muestra la diferencia, sin modificar nada todavía.*

1. EL CÓDIGO

Lo que **declaras** en los
ficheros `.tf` (el estado
deseado)

2. EL ESTADO

Lo último que OpenTofu
sabe que existe
(`terraform.tfstate`)

3. LA REALIDAD

Lo que **hay de verdad** en la
plataforma (puede haber
cambiado a mano: *drift*)



`plan` es la red de seguridad antes de aplicar: revisa siempre su salida antes de ejecutar `apply`.

04

OpenTofu + libvirt

Máquinas virtuales con KVM y cloud-init

El provider `libvirt`

```
terraform {  
  required_providers {  
    libvirt = {  
      source  = "dmacvicar/libvirt"  
      version = "0.8.3"  
    }  
  }  
}  
  
provider "libvirt" {  
  uri = "qemu:///system"  
}
```

- `uri` — la conexión a libvirt, la misma que `virsh -c qemu:///system`
- Versión **fijada** a la `0.8.3`: desde la `0.9` se reescribió con otra sintaxis

Imágenes cloud y el pool

Las **imágenes cloud** son sistemas ya instalados, mínimos, en formato `qcow2` y preparados para configurarse con cloud-init en el primer arranque.

```
cd /var/lib/libvirt/images          # directorio del pool "default"
sudo wget <url de la imagen> -O debian13-base.qcow2
sudo qemu-img resize debian13-base.qcow2 10G
sudo virsh pool-refresh default     # que libvirt vea el fichero nuevo
```

- Se descargan **una vez** y se usan como **imagen base** de todas las máquinas
- En los ejemplos: `debian13-base.qcow2` (Debian 13) y `ubuntu2604-base.qcow2` (Ubuntu 26.04)

Los discos: `libvirt_volume`

Clon ligero de la imagen base (*backing store*: solo guarda los cambios) y disco extra vacío de 1 GB:

```
resource "libvirt_volume" "server1-disk" {
  name          = "server1.qcow2"
  pool          = var.libvirt_pool_name
  base_volume_name = var.base_image
  base_volume_pool = var.libvirt_pool_name
  format        = "qcow2"
}

resource "libvirt_volume" "server1-extra" {
  name     = "server1-extra.qcow2"
  pool     = var.libvirt_pool_name
  format   = "qcow2"
  size     = 1 * 1024 * 1024 * 1024
}
```

cloud-init

Configura la máquina en su **primer arranque**, a partir de dos ficheros:

user-data : usuarios, claves, paquetes...

```
#cloud-config
hostname: server1
users:
  - name: debian
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh-authorized-keys:
      - ssh-ed25519 AAAA... tu clave
packages:
  - qemu-guest-agent
```

network-config : la red (netplan)

```
network:
  version: 2
  ethernets:
    ens3:
      dhcp4: true
    ens4:
      addresses: ["10.0.0.1/24"]
```

La máquina virtual: `libvirt_domain`

```
# Disco ISO con los ficheros de cloud-init
resource "libvirt_cloudinit_disk" "server1-cloudinit" {
  name          = "server1-cloudinit.iso"
  pool          = var.libvirt_pool_name
  user_data     = file("${path.module}/cloud-init/user-data1.yaml")
  network_config = file("${path.module}/cloud-init/network-config1.yaml")
}

resource "libvirt_domain" "server1" {
  name      = "server1"
  memory    = 1024
  vcpu      = 2
  qemu_agent = true
  network_interface { network_name = "default" }
  disk { volume_id = libvirt_volume.server1-disk.id }
  cloudinit = libvirt_cloudinit_disk.server1-cloudinit.id
}
```


Redes de libvirt

TIPO	DEFINICIÓN	¿EL ANFITRIÓN TIENE IP?	¿SALIDA AL EXTERIOR?
NAT	mode = "nat" + addresses	Sí (la .1)	Sí
Aislada	mode = "none" + addresses	Sí (la .1)	No
Muy aislada	mode = "none" , sin addresses	No	No

```
resource "libvirt_network" "nat-dhcp" {
  name      = "nat-dhcp"
  mode      = "nat"
  addresses = ["192.168.100.0/24"]
  dhcp { enabled = true }
  autostart = true
}
```

Conectar la máquina a las redes

RED QUE NO GESTIONA OPENTOFU

Por su **nombre** (por ejemplo, `default`):

```
network_interface {  
  network_name    = "default"  
  wait_for_lease = true  
}
```

RED CREADA POR OPENTOFU

Por su **id**:

```
network_interface {  
  network_id      = libvirt_network.nat-dhcp.id  
  wait_for_lease = true  
}
```

- `wait_for_lease = true` — esperar a que el DHCP le dé IP: **solo** en redes con DHCP
- Cada `network_interface` es una interfaz más (`ens3`, `ens4` ...), que hay que configurar en el `network-config`

Las IP en el `output`

```
output "server1" {  
  value = {  
    ip1 = try(libvirt_domain.server1.network_interface[0].addresses[0], "No disponible")  
  }  
}
```

- Sin agente, OpenTofu solo conoce las IP que reparte el **DHCP** de libvirt
- Con `qemu_agent = true`, se las pregunta al agente `qemu-guest-agent` de la máquina (lo instala cloud-init): también las **estáticas**
- `try(..., "No disponible")` — si aún no hay IP, el `output` no da error
- Si sale «No disponible», el agente aún no funcionaba: `tofu refresh` y `tofu output`

Del `output` al inventario de Ansible

`inventario.tf`

```
resource "local_file" "inventario" {
  filename = "${path.module}/hosts"
  content = templatefile(
    "${path.module}/inventario.tftpl", {
      ip_web = try(libvirt_domain.web
        .network_interface[0].addresses[0], "")
    })
}
```

`inventario.tftpl` (la plantilla)

```
[servidores_web]
web ansible_host=${ip_web} ansible_user=debian
```

- `templatefile` rellena la plantilla con las IP · `local_file` la escribe en tu equipo (provider `hashicorp/local`: repite `tofu init`)
- Cada `apply` genera el inventario con las IP correctas: **primero OpenTofu, después Ansible**

05

Resumen

Problemas frecuentes

Problemas frecuentes

SÍNTOMA	CAUSAS HABITUALES
<code>storage volume not found</code>	Falta <code>virsh pool-refresh default</code> · el nombre de la imagen no coincide con <code>variables.tf</code>
La red ya existe o se solapa	No se ha destruido el escenario anterior (mismo nombre o rango)
<code>Permission denied (publickey)</code>	No has puesto tu clave pública en el <code>user-data</code>
El <code>apply</code> se queda esperando	<code>wait_for_lease</code> en una red sin DHCP
Una interfaz sin IP	Falta en el <code>network-config</code> · nombre de interfaz equivocado
cloud-init termina con error	La máquina no tiene salida al exterior y se instalan paquetes
Errores de sintaxis del provider	Se ha cambiado la versión fijada (<code>0.8.3</code>) · documentación de la <code>0.9</code>

Antes de aplicar: `tofu validate` y `tofu plan`. Dentro de la máquina: `cloud-init status --long`.

PI · UNIDAD 1 — SOFTWARE DE ORQUESTACIÓN

¡Gracias!

Software de Orquestación · OpenTofu



José Domingo Muñoz



IES Gonzalo Nazareno · Dos Hermanas



PI